
⭐ 1. Managing “Multiple Inventories” in Ansible

Ansible allows you to use **more than one inventory file at the same time**.

If you point Ansible to a **directory** instead of a single file:

-i inventories/

Then Ansible will:

- Load **all static inventory files** inside that directory
- Execute **all dynamic inventory scripts** inside that directory
- Combine everything into **one final inventory**

This is extremely useful when you have:

- Cloud hosts (dynamic inventory)
- On-prem hosts (static inventory)
- Group definitions split across files
- Different teams managing different inventory files

⭐ 2. Example Directory Structure

inventories/

├— cloud-east.yml

├— cloud-west.yml

├— onprem.ini

├— dynamic-aws-inventory.py (executable → dynamic)

└— dynamic-azure-inventory.sh (executable → dynamic)

Ansible will combine **all** of these into one inventory.

⭐ 3. Static + Dynamic Inventory Example

File 1: cloud-east.ini

[cloud-east]

(Empty group — dynamic inventory will fill it)

File 2: servers.ini

[servers]

test.demo.example.com

[servers:children]

cloud-east

Dynamic inventory output (AWS script)

```
{
  "cloud-east": {
    "hosts": [
      "ec2-1.example.com",
      "ec2-2.example.com"
    ]
  }
}
```

Final combined inventory

[servers]

test.demo.example.com

ec2-1.example.com

ec2-2.example.com

⭐ 4. Why You MUST Declare Empty Groups

This is the part people misunderstand.

If you write:

[servers:children]

cloud-east

Then **you MUST also define the group cloud-east somewhere**, even if it's empty:

[cloud-east]

Why?

Because:

- Ansible loads inventory files in **alphabetical order**
- Dynamic inventory may load **after** static inventory
- If cloud-east is not defined yet, Ansible throws an error

✓ Correct (safe)

[cloud-east]

[servers]

test.demo.example.com

[servers:children]

cloud-east

❌ Incorrect (unsafe)

[servers]

test.demo.example.com

[servers:children]

cloud-east

This fails if the dynamic inventory hasn't loaded yet.

⭐ 5. Why Order Matters

Ansible **does not guarantee** the order of inventory file loading.

Currently:

- Files are loaded **alphabetically**
- But this is **not guaranteed** in future versions

So you must write inventories that are **self-consistent**, meaning:

- Every group referenced must exist
- No file should depend on another file loading first

⭐ 6. Ignoring Files in Inventory Directory

Ansible ignores files with certain extensions:

Ignored by default:

- .ini~
- .bak
- .orig
- .pyc
- .pyo
- .swp

You can control this in ansible.cfg:

[defaults]

inventory_ignore_extensions = .bak, .tmp, .old

This is useful when:

- You keep backups in the inventory directory
- You use editors that create temporary files

⭐ 7. Full Practical Example

Directory:

inventories/

├── 01-base.ini

├── 02-cloud.ini

└── 03-groups.ini

01-base.ini

[onprem]

server1

server2

02-cloud.ini

[cloud-east]

(Dynamic inventory will populate this)

03-groups.ini

[allservers:children]

onprem

cloud-east

Resulting inventory:

onprem:

  server1

  server2

cloud-east:

  ec2-1

  ec2-2

allservers:

  server1

server2

ec2-1

ec2-2

⭐ 8. Key Rules to Remember

Rule	Why it matters
Always define empty groups	Prevents errors when dynamic inventory loads later
Do not depend on file order	Ansible does not guarantee order
Keep inventories self-contained	Avoid cross-file dependency failures
Use inventory_ignore_extensions	Prevents accidental loading of backup/temp files

---